Cafe Sales Data Analysis

A Step-by-Step Data Cleaning & Exploratory Analysis Report

Prepared by **Feriel**
Data Analyst

Dataset: 10,000 raw cafe transaction records
Tools: Python (Pandas, NumPy), Matplotlib, Seaborn, Google Colab

Contents

1 Introduction

This report documents a complete, step-by-step exploratory data analysis (EDA) of a cafe sales dataset containing 10,000 raw transaction records. The dataset was deliberately "dirty": every column contained a mix of valid entries and placeholder junk values (`"UNKNOWN"`, `"ERROR"`), and every column was typed as a generic `object` rather than a proper numeric or date type.

The goal of this project was to simulate a realistic analyst workflow:

- Assess data quality before touching a single value
- Design and apply a transparent, reproducible cleaning pipeline
- Explore each variable individually before looking at relationships
- Extract sales, product, location, payment, and time-based insights
- Translate findings into business-relevant recommendations

Tools used: Python 3, Pandas, NumPy, Matplotlib, Seaborn, Google Colab.

2 Dataset Overview

The raw dataset consists of 10,000 rows and 8 columns, summarized below.

Column	Type (raw)	Description
Transaction ID	object	Unique identifier for each sale (no duplicates, no missing values)
Item	object	Product sold (Coffee, Tea, Cake, Cookie, Salad, Sandwich, Smoothie, Juice)
Quantity	object	Number of units sold in the transaction (1–5)
Price Per Unit	object	Unit price of the item (0.5–5.0)
Total Spent	object	Quantity × Price Per Unit
Payment Method	object	Cash, Credit Card, or Digital Wallet
Location	object	In-store or Takeaway
Transaction Date	object	Date of the transaction (2023)

Every column was initially stored as `object` (text) because placeholder strings such as `"UNKNOWN"` and `"ERROR"` were mixed in with numeric and date values, which prevented Pandas from inferring the correct type automatically.

3 Data Quality Assessment

Before any cleaning, a full audit of missing and invalid values was performed using `df.isna().sum()` combined with `df[col].unique()` on each column, to detect placeholder junk values that are not recognized as missing by default.

3.1 Missing Values — Before Cleaning

Column	Missing / Invalid Values
Transaction ID	0
Item	333
Quantity	138
Price Per Unit	179
Total Spent	173
Payment Method	2,579
Location	3,265
Transaction Date	159

Note: these counts reflect only values recorded as `NaN` at the raw stage — placeholder strings like `"UNKNOWN"` and `"ERROR"` were still counted as valid strings at this point, meaning the true proportion of unusable data was significantly higher.

3.2 Duplicate Records

A check with `df.duplicated().sum()` confirmed **0 duplicate rows** in the raw dataset.

4 Step-by-Step Cleaning Walkthrough

Each step below was applied in sequence, with the reasoning behind the chosen technique.

Step 1 — Standardize placeholder values to NaN

The strings `"UNKNOWN"` and `"ERROR"` appear across *Item*, *Payment Method*, *Location*, and *Transaction Date*. These were replaced with proper `NaN` values so that Pandas' missing-value tools (`isna()`, `fillna()`, `dropna()`) could handle them correctly.

> *Why:* leaving them as text would silently corrupt any groupby or aggregation (e.g. `"UNKNOWN"` would be treated as a real location).

Step 2 — Convert numeric columns

Quantity and *Price Per Unit* were converted using `pd.to_numeric(..., errors='coerce')`, which turns any value that cannot be parsed as a number into `NaN` instead of raising an error.

> *Why:* `coerce` is safer than manually filtering invalid rows one by one, and preserves the row for later imputation rather than deleting it immediately.

Step 3 — Impute missing Quantity

Missing *Quantity* values were filled with the column mean (≈ 3.03).

> *Why:* mean imputation is a reasonable default for a roughly symmetric, bounded numeric variable (1–5) when the missing rate is moderate and no stronger signal (e.g. per-item average) is required for this stage of analysis.

Step 4 — Recompute Total Spent

Total Spent was recalculated as *Quantity* × *Price Per Unit* rather than trusting the original column.

Why: the original *Total Spent* column had its own independent missing/corrupted values; recomputing it from two already-cleaned columns guarantees internal consistency across the dataset.

Step 5 — Parse Transaction Date

Transaction Date was converted with `pd.to_datetime(..., errors='coerce')`, enabling month-level and date-level aggregation later in the analysis.

Step 6 — Remove duplicates and empty rows

`drop_duplicates()` was applied (0 rows removed, confirming the earlier audit), followed by `dropna(thresh=6)`, which removes any row with fewer than 6 non-null values across its 8 columns — i.e. rows too incomplete to be analytically useful.

4.1 Missing Values — After Cleaning

Column	Remaining Missing Values
Transaction ID	0
Item	785
Quantity	0
Price Per Unit	191
Total Spent	191
Payment Method	2,835
Location	3,569
Transaction Date	352

The dataset went from **10,000** to **9,475** rows after removing duplicates and near-empty records. *Quantity* is now fully populated (0 missing) thanks to mean imputation; the remaining gaps in categorical columns (*Payment Method*, *Location*) were deliberately left as `NaN` rather than imputed, since inventing a payment method or location would introduce false information into the analysis.

5 Univariate Analysis

5.1 Item Distribution

Item	Number of Orders
Coffee	1,132
(7 other items)	remainder of 9,475
Tea	1,037 (lowest)

Coffee is the most frequently ordered item overall, while Tea is ordered least often. Eight distinct products appear in the dataset: Coffee, Tea, Cake, Cookie, Salad, Sandwich, Smoothie, and Juice.

5.2 Payment Method Distribution

Payment Method	Number of Transactions
Digital Wallet	2,233 (most common)
Cash	—
Credit Card	—

Three payment methods are used across the dataset, with Digital Wallet slightly ahead of Cash and Credit Card in raw transaction count.

5.3 Location Distribution

Two location types are recorded: **In-store** and **Takeaway**, with a large number of missing values (3,569 after cleaning) — a limitation discussed in Section 10.

5.4 Quantity Distribution

Quantity values range from 1 to 5 units per transaction, with an average of **3.03** units after imputation.

6 Key Sales Metrics

Metric	Value
Total Revenue	82,900.80
Average Transaction Value	8.93
Highest Order Value	25.00
Lowest Order Value	1.00
Average Quantity per Order	3.03
Valid Transactions Analyzed	9,475

7 Bivariate & Correlation Analysis

	Quantity	Total Spent
Quantity	1.00	0.70
Total Spent	0.70	1.00

	Price/Unit	Total Spent
Price Per Unit	1.00	0.66
Total Spent	0.66	1.00

Both *Quantity* and *Price Per Unit* show a **moderate positive correlation** with *Total Spent* ($r = 0.70$ and $r = 0.66$ respectively). Neither variable alone fully explains total spend, which is expected since *Total Spent* is a direct product of the two — the moderate (rather than very high) correlation reflects the wide range of combinations between quantity and price across transactions.

8 Visual Dashboard

Figure 1 presents six complementary views of the cleaned dataset: the overall distribution of sale values, the quantity–spend relationship, revenue by location, revenue by item, the monthly sales trend, and the payment method split.

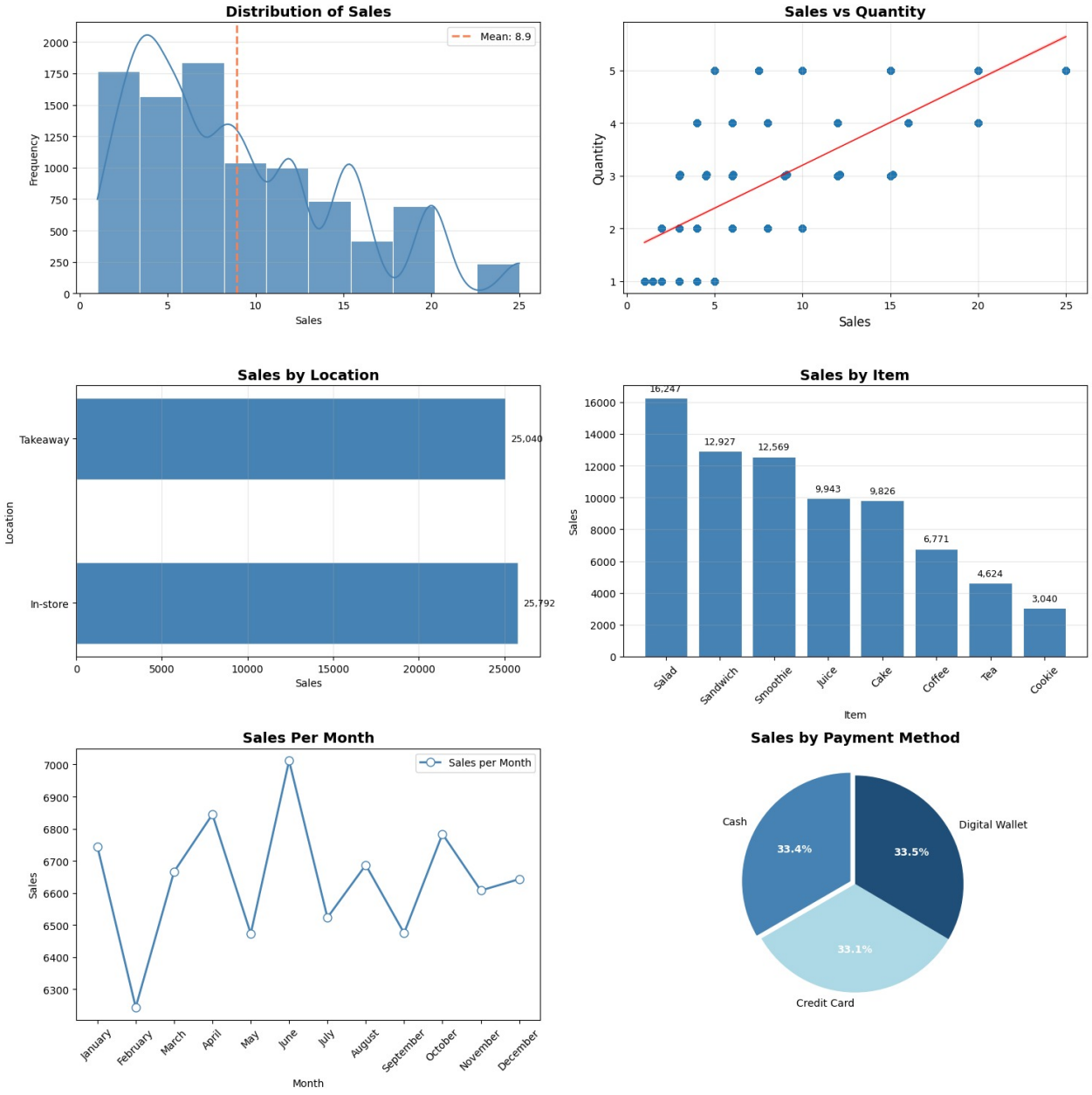

Figure 1: Cafe Sales Analysis Dashboard

9 Key Insights

9.1 Product Performance

- **Salad** is the top revenue-generating item (16,247.26), followed by **Sandwich** (12,926.83) and **Smoothie** (12,568.78).
- **Coffee** is the best-selling item by volume (1,132 orders) yet ranks sixth in total revenue (6,771.36) — a sign of its lower unit price relative to items like Salad.
- **Cookie** generates the lowest revenue overall (3,040.42).
- By quantity sold, **Coffee** again leads (3,465.68 units), closely followed by Juice and Cake — confirming Coffee's high order frequency even though it is not the top revenue driver.

9.2 Location & Payment

- Revenue is nearly evenly split between **In-store** (25,792.39) and **Takeaway** (25,039.96), a difference of less than 3%.
- Payment methods are almost perfectly balanced: **Digital Wallet** (33.5%, 19,415.28), **Cash** (33.4%, 19,348.94), and **Credit Card** (33.1%, 19,220.71).
- Average spend per transaction is highest for Cash payments (9.05) and lowest for Digital Wallet (8.94), though the gap is small enough to be practically negligible.

9.3 Time Trends

- **June** recorded the highest monthly revenue (7,011.90), while **February** recorded the lowest (6,244.06).
- Excluding these two months, revenue stays within a narrow band (roughly 6,470–6,850), suggesting the cafe's sales are largely stable across the year with limited strong seasonality.

9.4 Correlations

- Quantity and Total Spent: $r = 0.70$ (moderate positive).
- Price Per Unit and Total Spent: $r = 0.66$ (moderate positive).
- Both relationships confirm expected purchasing behavior without revealing a single dominant driver of spend.

10 Limitations

- **No customer identifier** exists in the dataset, so repeat-customer behavior, loyalty patterns, and customer segmentation could not be analyzed.
- **High missingness in Location (3,569) and Payment Method (2,835)** after cleaning means location- and payment-based conclusions are drawn from roughly 60–70% of the full dataset; the true distribution could differ slightly if the missing data is not random.
- **Mean imputation for Quantity** assumes missing values behave like the average transaction; it does not account for possible per-item differences in typical order size.
- **No cost or margin data** is available, so revenue figures reflect sales volume only and cannot be translated directly into profitability.

11 Recommendations

1. **Review Coffee's pricing strategy.** It is the highest-volume item but only the sixth highest in revenue — a small, tested price increase could raise revenue without significantly affecting demand.
2. **Investigate the February dip and June peak** in monthly sales to identify whether external factors (promotions, weather, holidays) explain the swing, and replicate any favorable conditions found in June.
3. **Fix data collection for Location and Payment Method**, which have the highest missing rates; these are likely operational/POS logging issues rather than genuine unknowns, and resolving them would substantially strengthen future analysis.
4. **Introduce a customer identifier** in future data collection to enable loyalty and repeat-purchase analysis, which is currently impossible with this dataset.

12 Conclusion

This project walked through a complete analyst workflow on a deliberately messy dataset: auditing data quality, designing a transparent and justified cleaning pipeline, analyzing each variable individually, then jointly, and translating the results into concrete business recommendations. The cafe's revenue is well-balanced across location and payment method, with moderate seasonality and a clear opportunity to reconsider pricing for high-volume, lower-margin items such as Coffee. The most valuable next step for the business would be improving data capture quality, particularly for Location and Payment Method, and introducing a customer identifier to unlock deeper behavioral analysis.

A Appendix — Full Python Code

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns

df = pd.read_csv("/content/sample_data/dirty_cafe_sales.csv")

# --- Cleaning ---
cols_to_clean = ['Item', 'Payment Method', 'Location', 'Transaction Date']
for col in cols_to_clean:
    df[col] = df[col].replace(['UNKNOWN', 'ERROR'], np.nan)

df['Quantity'] = pd.to_numeric(df['Quantity'], errors='coerce')
df['Price Per Unit'] = pd.to_numeric(df['Price Per Unit'], errors='coerce')
df['Quantity'] = df['Quantity'].fillna(df['Quantity'].mean())

df['Total Spent'] = df['Quantity'] * df['Price Per Unit']
df['Total Spent'] = pd.to_numeric(df['Total Spent'], errors='coerce')

df['Transaction Date'] = pd.to_datetime(df['Transaction Date'], errors='coerce')

df = df.drop_duplicates()
df = df.dropna(thresh=6)

# --- Metrics ---
total_sales = df['Total Spent'].sum()
avg_transaction = df['Total Spent'].mean()
sales_by_item = df.groupby('Item')['Total Spent'].sum().sort_values(ascending=False)
sales_by_payment = df.groupby('Payment Method')['Total Spent'].sum()
sales_by_location = df.groupby('Location')['Total Spent'].sum()

months = df['Transaction Date'].dt.month_name()
sales_by_month = df.groupby(months)['Total Spent'].sum()

corr_qty_spent = df[['Quantity', 'Total Spent']].corr()
corr_price_spent = df[['Price Per Unit', 'Total Spent']].corr()
```